휴리스틱 알고리즘

가까운 도시 먼저 가기처럼 간단한 선택 규칙으로 해를 구하거나 고칩니다.

먼저 작은 예로 살펴봅니다

휴리스틱

가까운 도시 먼저 가기처럼 간단한 선택 규칙으로 해를 구하거나 고칩니다.

무엇을 반환하나요?

문제에서 정한 규칙을 그대로 실행한 결과입니다. 규칙과 동률 조건을 먼저 읽습니다.

11-1 격자에서 맨해튼 거리

구할 것

격자에서 가로·세로로만 이동할 때 필요한 칸 수를 구합니다.

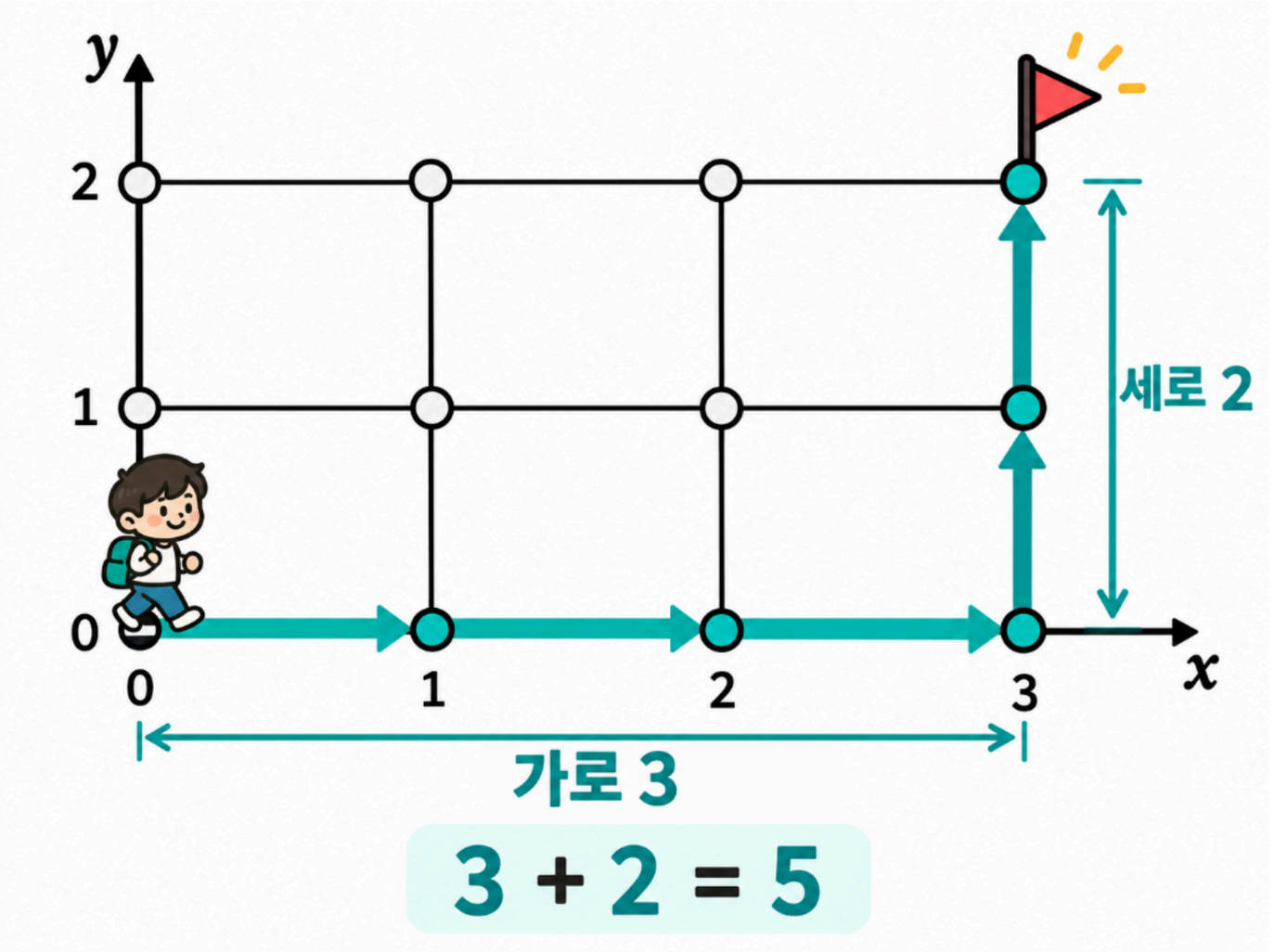
작은 예

(0,0)에서 (3,2)로 갑니다.
가로 세 칸, 세로 두 칸을 움직여야 합니다.

반환할 값

장애물 없이 상하좌우로만 움직이므로 최소 3+2=5를 반환합니다.

11-1 그림으로 보는 예



오른쪽으로 세 칸
위쪽으로 두 칸
순서가 달라도
총 다섯 번 이동

11-1 하나씩 계산한 결과

출발	도착	가로 차이	세로 차이	합
(0,0)	(3,2)	3	2	5
(2,3)	(2,3)	0	0	0
(2,–1)	(–2,2)	4	3	7

이 거리를 맨해튼 거리라고 부릅니다.

11-1 계산 순서

가로 차이

x좌표를 빼고 절댓값을 구합니다.

세로 차이

y좌표도 같은 방법으로 계산합니다.

합

가로 이동 수와 세로 이동 수를 더합니다.

11-1 변수와 코드의 뜻

코드	뜻
x1-x2	방향에 따라 양수나 음수가 나오는 가로 차이
abs(x1-x2)	부호를 없앤 가로 이동 칸 수
abs(y1-y2)	세로 이동 칸 수

절댓값 abs는 음수의 부호를 없애 거리처럼 양수로 읽게 합니다.

11-1 함수에 들어오는 값

함수

manhattan(x1, y1, x2, y2)

입력 조건

각 좌표는 -10,000~10,000인 정수입니다. 장애물이 없습니다.

11-1 Python

```python
def manhattan(x1, y1, x2, y2):
    return abs(x1-x2) + abs(y1-y2)
```

시간 O(1), 추가 공간 O(1)

11-1 C

```
int manhattan(int x1, int y1, int x2, int y2) {
    int dx=x1-x2, dy=y1-y2;
    if (dx<0) dx=-dx;
    if (dy<0) dy=-dy;
    return dx+dy;
}
```

시간 O(1), 추가 공간 O(1)

11-1 다른 입력도 확인합니다

상황	답	이유
출발점과 도착점 같음	0	이동할 필요 없음
(0,0)에서 (1,1)	2	대각선 한 번 이동은 허용하지 않음
음수 좌표 포함	차이의 절댓값 합	좌표 자체의 부호와 무관

절댓값 abs는 음수의 부호를 없애 거리처럼 양수로 읽게 합니다.

11-2 가장 가까운 도시부터 방문

구할 것

현재 위치에서 가장 가까운 미방문 도시로 계속 이동합니다.

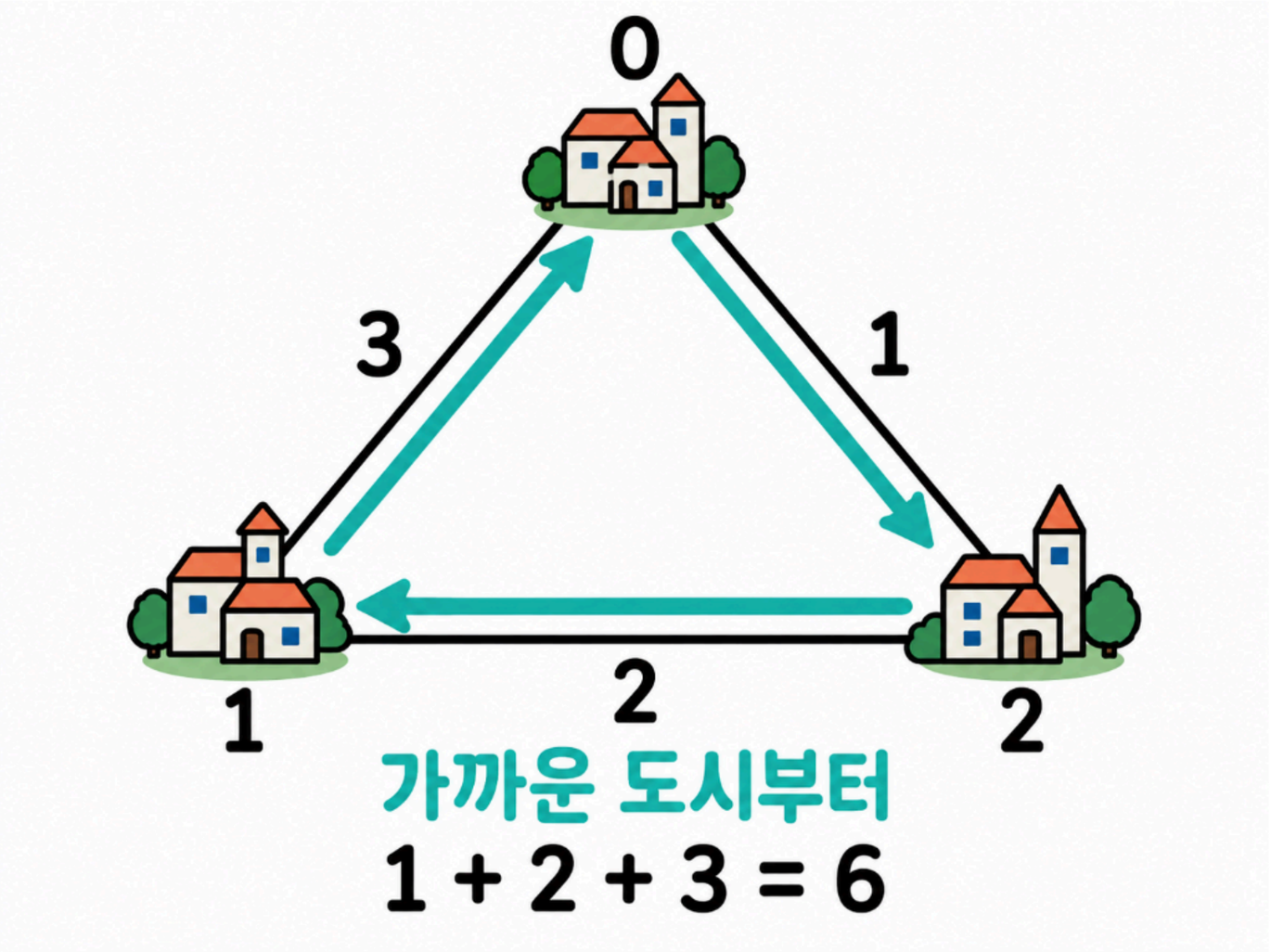
작은 예

0에서 1까지는 3, 2까지는 1입니다. 먼저 2로 갑니다.
2에서 남은 1로 가는 비용은 2, 1에서 0으로 돌아오는 비용은 3입니다.

반환할 값

이 규칙대로 이동한 비용 1+2+3=6을 반환합니다.

11-2 그림으로 보는 예



0에서는 2가 더 가까움
남은 도시는 1
마지막에 0으로 귀환
비용 합은 6

11-2 하나씩 계산한 결과

현재 도시	갈 수 있는 도시와 비용	선택	누적 합
0	1: 3, 2: 1	2	1
2	1: 2	1	3
1	방문 완료. 0으로 귀환: 3	0	6

모든 순회를 비교하지 않고, 그 순간 가장 가까운 도시를 하나 고릅니다.

11-2 계산 순서

출발 도시 방문 표시

0은 처음부터 방문한 도시로 표시합니다.

가까운 미방문 도시 선택

후보를 전부 읽어 비용이 가장 작은 곳을 고릅니다.

이동 후 갱신

비용을 더하고 방문 표시와 현재 위치를 바꿉니다.

11-2 변수와 코드의 뜻

변수	뜻
current	지금 있는 도시
used[v]	도시 v를 이미 방문했는지 표시
nxt	이번에 선택한 다음 도시
total+dist[current][0]	마지막 귀환까지 포함한 비용

지금 가까운 곳을 고르는 규칙입니다. 모든 경로 중 최소 비용과 다를 수 있습니다.

11-2 함수에 들어오는 값

함수

nearest_neighbor(dist)

입력 조건

도시 1~9개. 비용 0~100, 대각선 0. 방향별 비용은 달라도 됩니다.

11-2 Python (1/2)

```
def nearest_neighbor(dist):
    n = len(dist)
    used = [False]*n
    used[0] = True
    current = 0
    total = 0
    for step in range(n-1):
        nxt = -1
```

다음 쪽에서 이어집니다.

11-2 Python (2/2)

```
        for v in range(n):
            if not used[v]:
                if nxt == -1 or dist[current][v] < dist[current][nxt]:
                    nxt = v
        total += dist[current][nxt]
        used[nxt] = True
        current = nxt
    return total + dist[current][0]
```

시간 $O(n^2)$, 방문 배열 $O(n)$

11-2 C

```
int nearest_neighbor(const int dist[][9], int n) {
    int used[9]={0}, current=0, total=0;
    used[0]=1;
    for (int step=0; step<n-1; ++step) {
        int next=-1;
        for (int v=0; v<n; ++v)
            if (!used[v] && (next<0 || dist[current][v]<dist[current][next]))
                next=v;
        total+=dist[current][next];
        used[next]=1;
        current=next;
    }
    return total+dist[current][0];
}
```

시간 $O(n^2)$, 방문 배열 $O(n)$

11-2 다른 입력도 확인합니다

상황	답	확인할 점
도시 하나	0	추가 이동 없음
후보 두 곳의 비용이 같음	작은 번호 선택	동률 규칙
두 도시 왕복 비용 5,8	13	편도 비용만 반환하지 않음

지금 가까운 곳을 고르는 규칙입니다. 모든 경로 중 최소 비용과 다를 수 있습니다.

11-3 담금질 기법의 한 단계 수락 판단

구할 것

새 후보로 바꿀지 주어진 규칙에 따라 한 번 판단합니다.

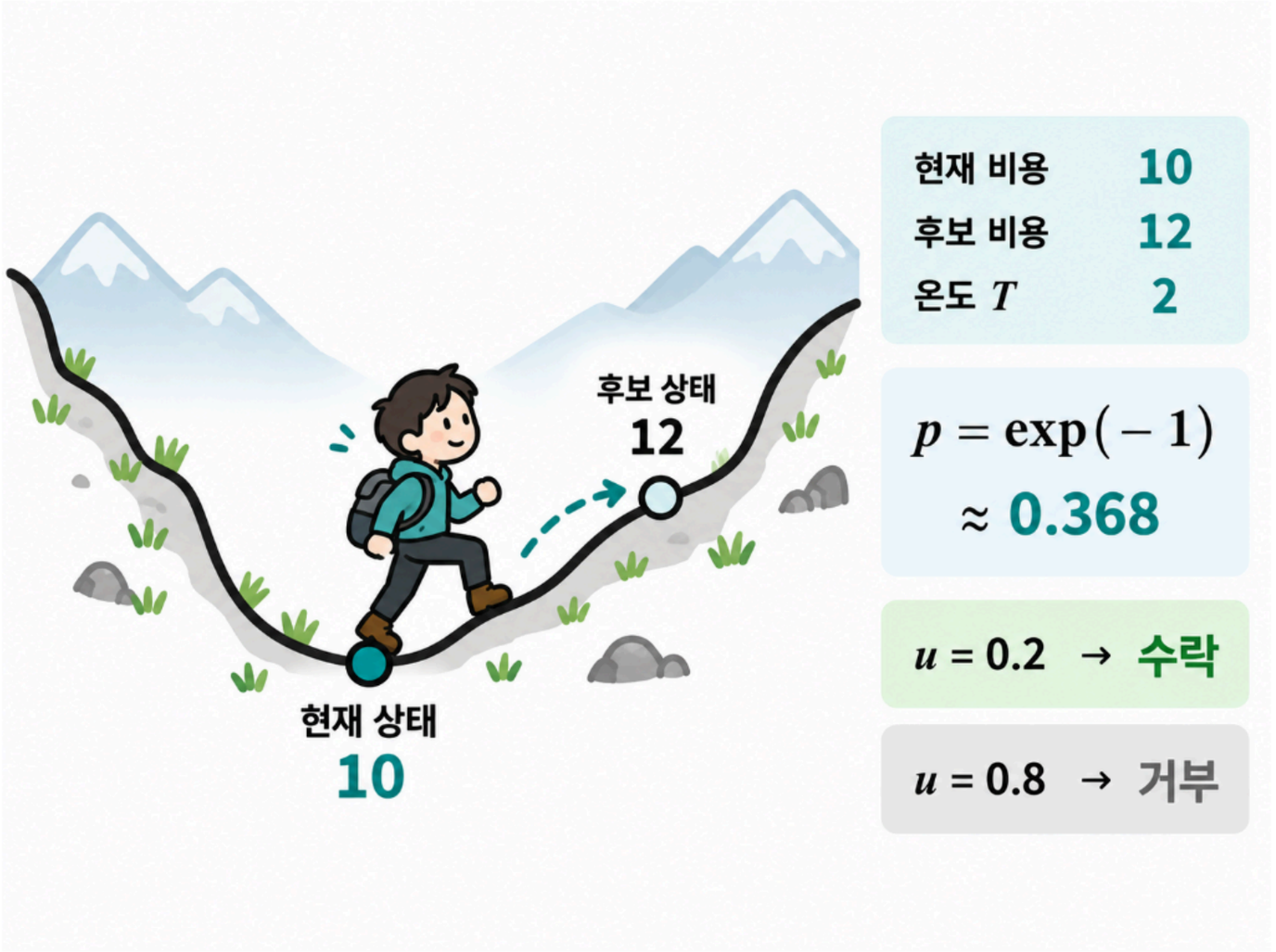
작은 예

현재 비용 10, 새 후보 비용 12, 온도 2입니다.
새 후보는 더 비싸지만 정해진 확률로 받아들일 수 있습니다.

반환할 값

이때 확률은 약 0.368입니다. 입력 u가 0.2면 수락 1, 0.8이면 거부 0입니다.

11-3 그림으로 보는 예



현재 10, 후보 12
온도 2일 때
수락 확률 약 0.368
u=0.2이면 수락

11-3 하나씩 계산한 결과

후보의 상태	온도	판단
현재보다 작거나 같음	어떤 값이든	항상 수락 1
현재보다 큼	0	거부 0
현재보다 큼	0보다 큼	u와 수락 확률 비교

좋은 후보부터 먼저 처리하고, 더 나쁜 후보일 때만 확률을 계산합니다.

exp는 주어진 수학 함수입니다

예제에서는 $\exp(-1) \approx 0.368$이라는 계산값을 사용합니다.

Python

import math 뒤 math.exp(값)을 호출합니다.

C

#include <math.h> 뒤 exp(값)을 호출합니다.

아주 작은 확률

양의 온도에서는 확률이 0보다 크므로 u=0이면 수락합니다. 반올림으로 0이 되는 경우도 코드에서 먼저 처리합니다.

11-3 계산 순서

같거나 더 싼 후보

candidate≤current이면 1을 반환합니다.

온도 0

더 비싼 후보를 받아들이지 않아 0을 반환합니다.

확률 비교

exp((current−candidate)/temp)를 구해 u가 더 작으면 1입니다.

11-3 변수와 코드의 뜻

변수	예제에서의 뜻
current, candidate	현재 비용 10, 새 비용 12
temp	온도 2. 더 비싼 후보를 받아들이는 정도에 영향
probability	exp((10−12)/2)=exp(−1)≈0.368
u	주어진 0..1 사이 숫자. 새 난수를 만들지 않음

이런 수락 규칙을 쓰는 방법을 담금질 기법이라고 부릅니다.

11-3 함수에 들어오는 값

함수

accept_move(current, candidate, temp, u)

입력 조건

비용 0~10,000, 온도는 실수 0~1000, u는 0 이상 1 미만의 실수입니다.

11-3 Python

```
import math
def accept_move(current, candidate, temp, u):
    if candidate <= current: return 1
    if temp == 0: return 0
    if u == 0: return 1
    probability = math.exp((current-candidate)/temp)
    return int(u < probability)
```

기본 수학 연산을 상수 시간으로 볼 때 시간 O(1), 공간 O(1)

11-3 C

```
#include <math.h>
int accept_move(int current, int candidate, double temp, double u) {
    if (candidate<=current) return 1;
    if (temp==0) return 0;
    if (u==0) return 1;
    double probability=exp((current-candidate)/temp);
    return u<probability;
}
```

기본 수학 연산을 상수 시간으로 볼 때 시간 O(1), 공간 O(1)

11-3 다른 입력도 확인합니다

입력의 상황	답	이유
10에서 8로, 온도0	1	더 싼 후보
10에서 12로, 온도0	0	더 비싼 후보·온도0
10에서 12로, 온도2, u=0.2	1	0.2<0.368
같은 조건에서 u=0.8	0	0.8은 0.368보다 큼

이런 수락 규칙을 쓰는 방법을 담금질 기법이라고 부릅니다.

11-4 한 번의 도시 교환으로 순회 개선

구할 것

주어진 경로에서 도시 두 곳을 한 번 맞바꿔 비용을 줄입니다.

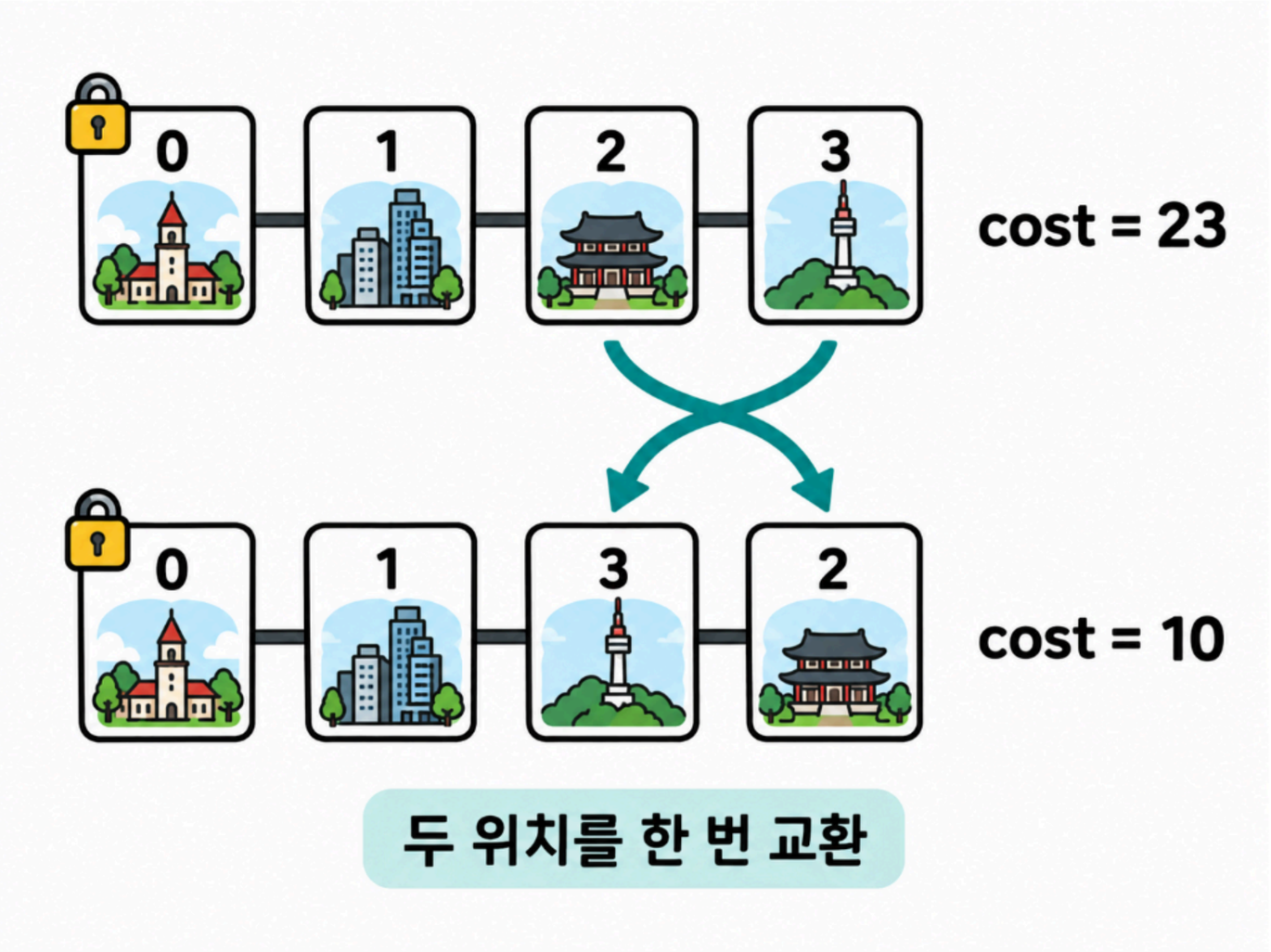
작은 예

처음 경로는 [0,1,2,3]이고 비용은 23입니다.
2와 3의 위치를 맞바꾸면 [0,1,3,2]가 되고 비용은 10입니다.

반환할 값

아무것도 바꾸지 않는 경우와 한 번 교환한 경우들을 비교해 최소 비용을 반환합니다.

11-4 그림으로 보는 예



첫 도시 0은 고정
나머지 두 위치 교환
[0,1,3,2]의 비용 10
각 후보 뒤 원상 복구

교환 예제의 이동 비용 표

아래 표와 원래 경로 [0,1,2,3]을 사용합니다.

출발	도착0	도착1	도착2	도착3
도시0	0	3	1	8
도시1	3	0	8	2
도시2	1	8	0	4
도시3	8	2	4	0

원래 비용은 3+8+4+8=23. 2와 3의 위치를 바꾸면 3+2+4+1=10입니다.

11-4 하나씩 계산한 결과

바꾸는 위치	경로	왕복 비용
안 바꿈	0,1,2,3	23
1번과 2번	0,2,1,3	19
1번과 3번	0,3,2,1	23
2번과 3번	0,1,3,2	10

가장 작은 비용은 10입니다. 모든 후보는 원래 경로에서 한 번만 바꿉니다.

11-4 계산 순서

원래 비용으로 시작

아무것도 바꾸지 않은 경로도 후보입니다.

두 위치 교환

첫 도시를 제외한 i,j 두 위치를 바꿔 비용을 계산합니다.

복구 후 다음 후보

다시 같은 두 위치를 바꿔 원래 순서로 돌립니다.

11-4 변수와 코드의 뜻

코드	뜻
best=cost()	원래 경로 비용 저장
i는 1부터	0번 위치의 출발 도시는 고정
j는 i+1부터	같은 두 위치를 중복 비교하지 않음
교환 코드가 두 번 등장	계산 전 교환, 계산 후 복구

개선된 경로에서 계속 바꾸는 방식이 아니라, 원래 경로의 한 번 교환만 비교합니다.

11-4 함수에 들어오는 값

함수

best_swap_cost(dist, route)

입력 조건

도시 1~9개. 비용 0~100, 대각선 0. 경로는 모든 도시를 한 번씩 포함하고 0에서 시작합니다.

11-4 Python (1/2)

```
def best_swap_cost(dist, route):
    route = route[:]
    n = len(route)
    def cost():
        total = 0
        for i in range(n):
            u = route[i]
            v = route[(i+1)%n]
            total += dist[u][v]
```

다음 쪽에서 이어집니다.

11-4 Python (2/2)

```
        return total
    best = cost()
    for i in range(1, n):
        for j in range(i+1, n):
            route[i], route[j] = route[j], route[i]
            best = min(best, cost())
            route[i], route[j] = route[j], route[i]
    return best
```

위치 쌍 $O(n^2)$ × 경로 비용 $O(n)$, 시간 $O(n^3)$, 복사 배열 $O(n)$

11-4 C (1/2)

```
static int swap_route_cost(const int d[][9], int n, const int r[]) {
    int sum=0;
    for (int i=0; i<n; ++i) sum+=d[r[i]][r[(i+1)%n]];
    return sum;
}
int best_swap_cost(const int dist[][9], int n, const int route[]) {
    int r[9];
    for (int i=0; i<n; ++i) r[i]=route[i];
    int best=swap_route_cost(dist,n,r);
    for (int i=1; i<n; ++i) for (int j=i+1; j<n; ++j) {
        int t=r[i];
```

다음 쪽에서 이어집니다.

11-4 C (2/2)

```
            r[i]=r[j];
            r[j]=t;
            int value=swap_route_cost(dist,n,r);
            if (value<best) best=value;
            t=r[i];
            r[i]=r[j];
            r[j]=t;
        }
        return best;
}
```

위치 쌍 $O(n^2)$ × 경로 비용 $O(n)$, 시간 $O(n^3)$, 복사 배열 $O(n)$

11-4 다른 입력도 확인합니다

상황	답	이유
도시 하나	0	교환할 두 위치 없음
도시 두 개	원래 왕복 비용	0을 고정하면 교환할 쌍 없음
교환하면 모두 더 비쌈	원래 비용	안 바꾸는 경우도 후보

개선된 경로에서 계속 바꾸는 방식이 아니라, 원래 경로의 한 번 교환만 비교합니다.

11장의 계산을 한 문장으로

문제	구하는 값
11-1	격자에서 가로·세로로만 이동할 때 필요한 칸 수를 구합니다.
11-2	현재 위치에서 가장 가까운 미방문 도시로 계속 이동합니다.
11-3	새 후보로 바꿀지 주어진 규칙에 따라 한 번 판단합니다.
11-4	주어진 경로에서 도시 두 곳을 한 번 맞바꿔 비용을 줄입니다.

가까운 도시 먼저 가기처럼 간단한 선택 규칙으로 해를 구하거나 고칩니다.